American International University- Bangladesh (AIUB)
Faculty of Engineering (EEE)

Course Name:	Microprocessor and Embedded Systems	**Course Code:**	COE3104
Semester:	Summer 2024-2025	**Sec:**	K
Lab Instructor:	Md. Ali Noor		

Experiment No:	04
Experiment Name:	Study of a Digital Timer using millis() function of Arduino to avoid problems associated with the delay()

Group Members	**Name**	**ID**
1.	Basudeb Kundu	23-50856-1
2.	Prohlad Chandra Das	23-50922-1
3.	Debashis Kumar Das	23-50953-1
4.	Indronill Dutta Nill	23-50974-1
5.	Nafiur Rahman Nirob	23-50991-1

Performance Date:	30-7-25	**Due Date:**	08-08-25

Marking Rubrics (to be filled by Lab Instructor)

Category	Proficient [6]	Good [4]	Acceptable [2]	Unacceptable [1]	Secured Marks
Theoretical Background, Methods & procedures sections	All information, measures and variables are provided and explained.	All Information provided that is sufficient, but more explanation is needed.	Most information correct, but some information may be missing or inaccurate.	Much information missing and/or inaccurate.	
Results	All of the criteria are met; results are described clearly and accurately;	Most criteria are met, but there may be some lack of clarity and/or incorrect information.	Experimental results don't match exactly with the theoretical values and/or analysis is unclear.	Experimental results are missing or incorrect;	
Discussion	Demonstrates thorough and sophisticated understanding. Conclusions drawn are appropriate for analyses;	Hypotheses are clearly stated, but some concluding statements not supported by data or data not well integrated.	Some hypotheses missing or misstated; conclusions not supported by data.	Conclusions don't match hypotheses, not supported by data; no integration of data from different sources.	
General formatting	Title page, placement of figures and figure captions, and other formatting issues all correct.	Minor errors in formatting.	Major errors and/or missing information.	Not proper style in text.	
Writing & organization	Writing is strong and easy to understand; ideas are fully elaborated and connected; effective transitions between sentences; no typographic, spelling, or grammatical errors.	Writing is clear and easy to understand; ideas are connected; effective transitions between sentences; minor typographic, spelling, or grammatical errors.	Most of the required criteria are met, but some lack of clarity, typographic, spelling, or grammatical errors are present.	Very unclear, many errors.	
Comments:				Total Marks (Out of):	

Title: Study of a Digital Timer using millis() function of Arduino to avoid problems associated with the delay() function.

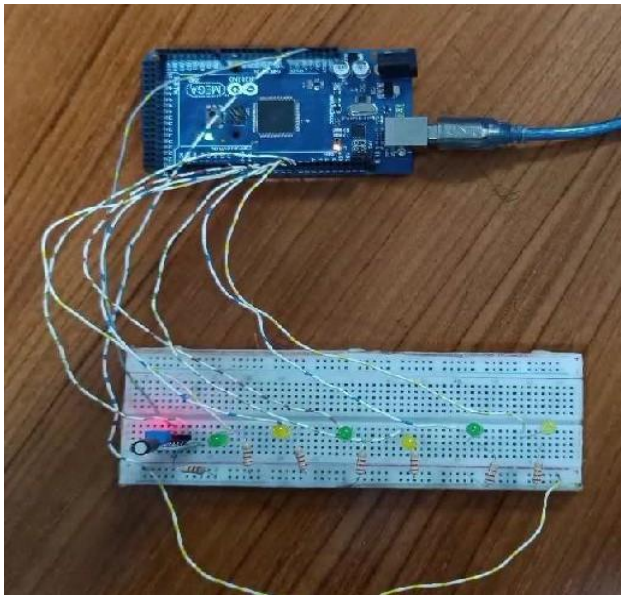
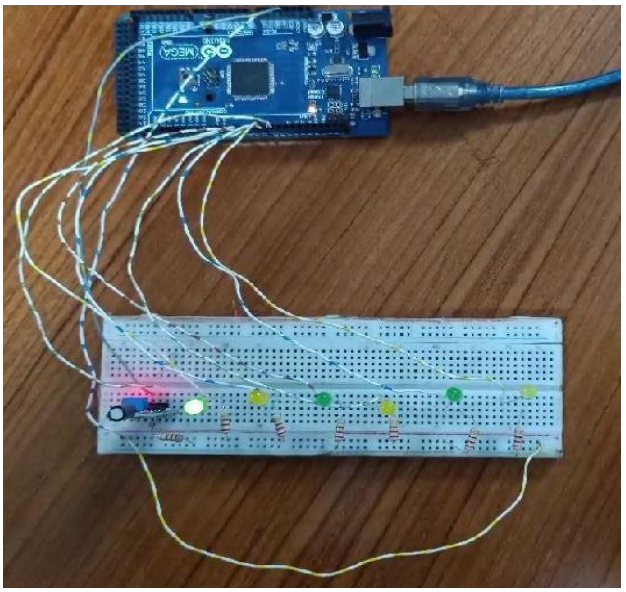
Objective:

In this project, we built a digital timer that turns on an LED every minute. Besides, we knew how long we were working on our projects by using Arduino's built-in Timer.

Apparatus:

1. Arduino Uno/Arduino Mega Microcontroller Board
2. Tilt sensor (One)
3. LEDs (Six)
4. Resistors (One 10 kilo ohm and Six 100 ohm)

Experimental Setup:

<table>
<tr><td colspan="2">Picture for LED control with tilt Sensor and millis() function</td></tr>
<tr><td> 
Figure 1: LED control with millis() function and tilt sensor</td><td> 
Figure 2: LED 1 on for 1 minute</td></tr>
</table>

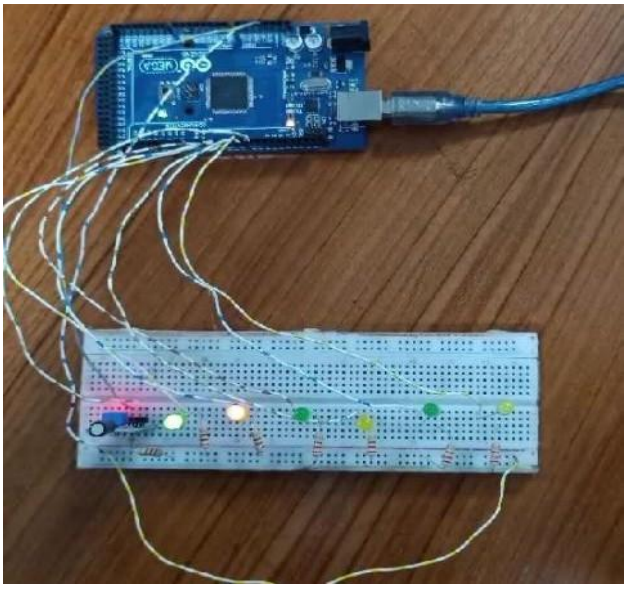


Figure 3: LED 2 on for 1 minute

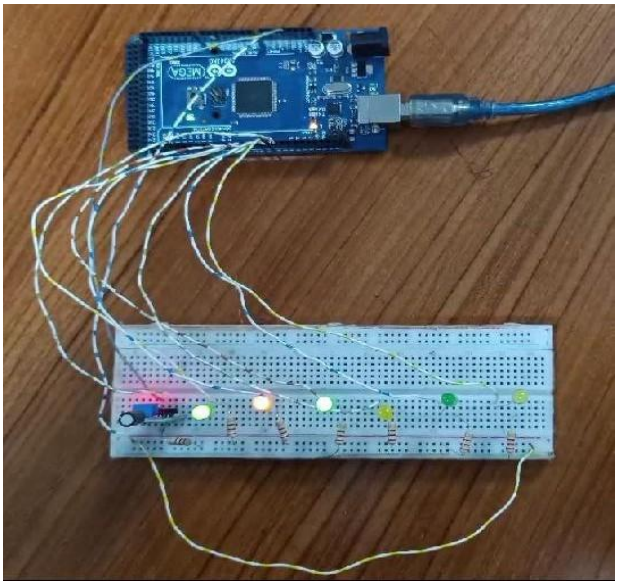


Figure 4: LED 3 on for 1 minute

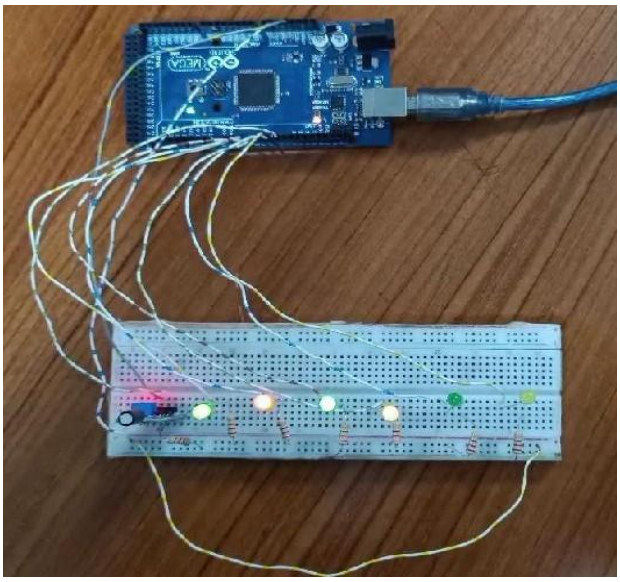


Figure 5: LED 4 on for 1 minute

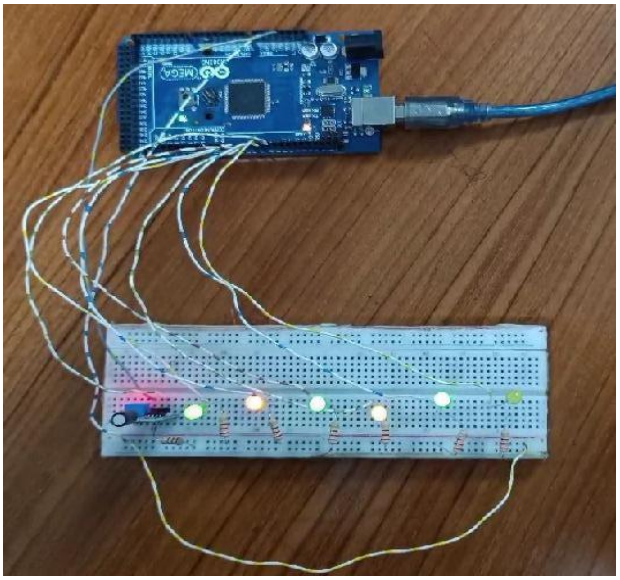


Figure 6: LED 5 on for 1 minute

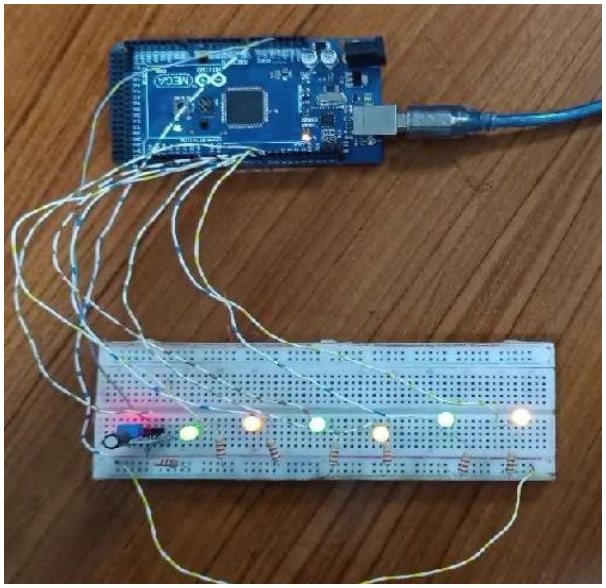


Figure 7: LED 6 on for 1 minute

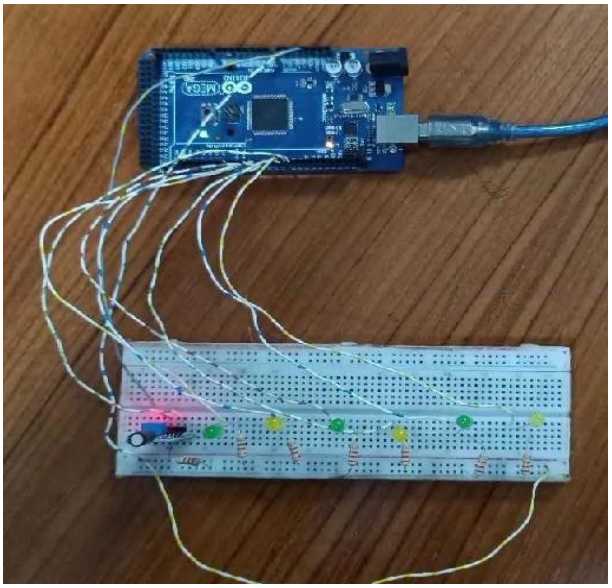


Figure 8: Reset LED by tilt sensor

Code:

Code for LED control with millis() function

```
const int SwitchPin = 8;
unsigned long PreviousTime = 0;
int SwitchState = 0;
int PrevSwitchState = 0;
int led = 2;
long interval = 60000;
void setup() {
        for (int x = 2; x < 8; x++)
        {
                pinMode(x, OUTPUT);
        }
        pinMode(SwitchPin, INPUT);
}

void loop() {
        unsigned long CurrentTime = millis();
        if (CurrentTime - PreviousTime > interval) {
                PreviousTime = CurrentTime;
                digitalWrite(led, HIGH);
```

```
            led++;
            if (led == 7) {
            }
        }
        SwitchState = digitalRead(SwitchPin);
        if (SwitchState != PrevSwitchState) {
            for (int x = 2 ; x < 8; x++)
            {
                digitalWrite(x, LOW);
            }
            led = 2;
            PreviousTime = CurrentTime;
        }
        PrevSwitchState = SwitchState;
}
```

Question Answer:

Simulation for LED control with Tilt sensor and millis() function

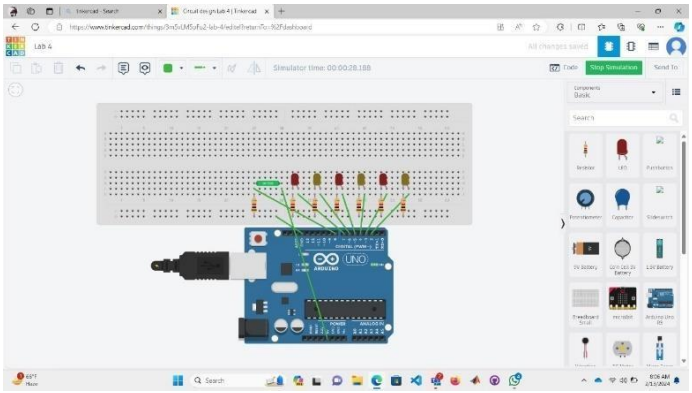


Figure 1: LED control by millis()function,tilt sensor

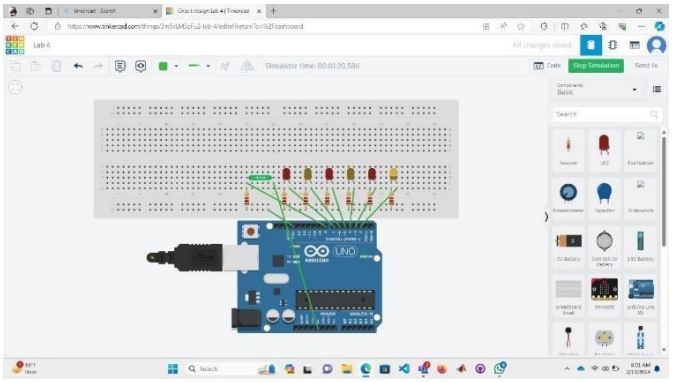


Figure 2: LED 1 on for 1 minute

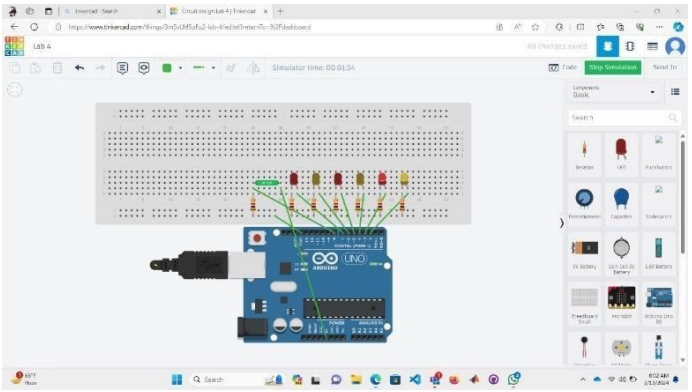


Figure 3: LED 2 on for 1 minute

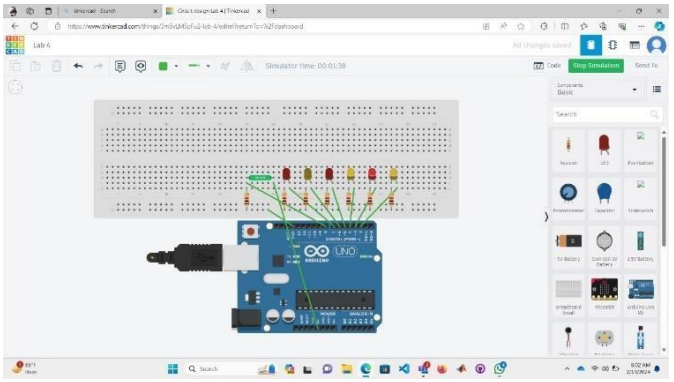


Figure 4: LED 3 on for 1 minute

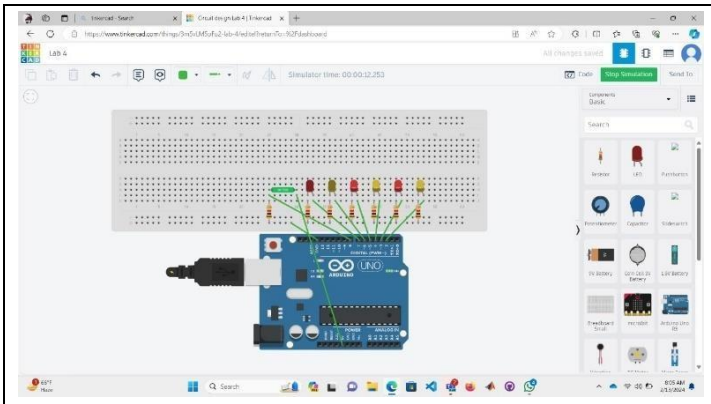


Figure 5: LED 4 on for 1 minute

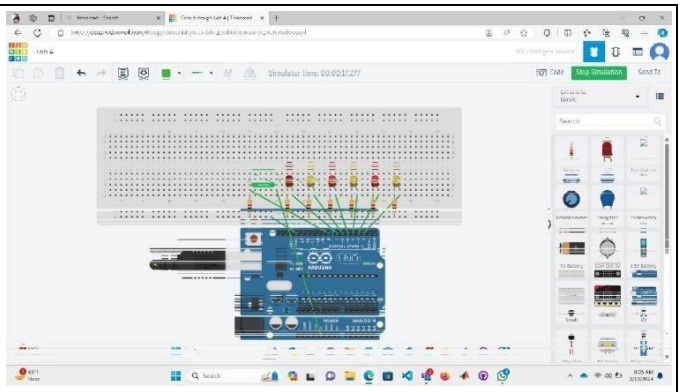


Figure 6: LED 5 on for 1 minute

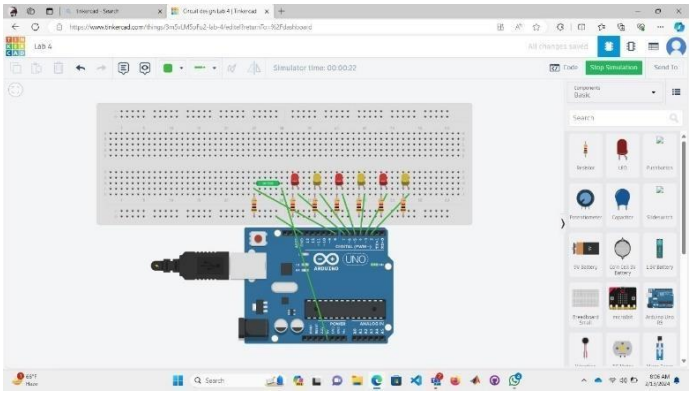


Figure 7: LED 6 on for 1 minute

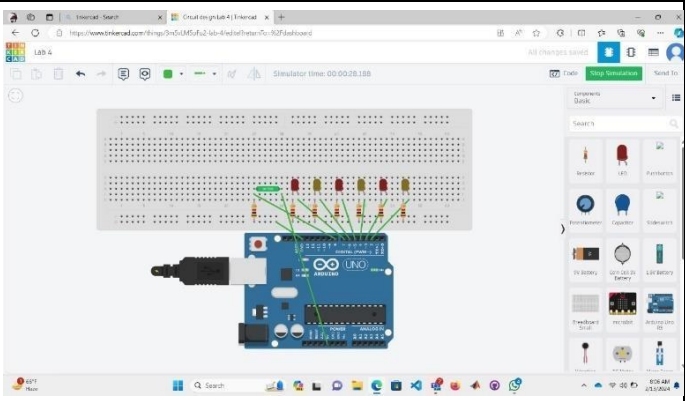


Figure 8: Reset LED by tilt sensor

Discussion and Conclusion:

The study aimed to design a digital timer using the millis() function in Arduino to overcome the limitations associated with the delay() function. The delay() function in Arduino can lead to blocking code execution, making it unsuitable for multitasking or time-sensitive applications. By employing millis(), a non-blocking alternative, the timer can perform other tasks while counting time accurately. The timer was integrated with a tilt sensor to initiate the timing process upon detecting movement. This enhances user interaction, making the timer more practical for various applications. The tilt sensor, acting as a trigger, eliminates the need for manual initiation, adding an element of automation. Moreover, by avoiding the delay() function, the system remains responsive, allowing for concurrent execution of tasks. This is crucial for real-world applications where multitasking is essential. The implementation ensures that the timer remains accurate, providing a reliable timekeeping mechanism.

In conclusion, the utilization of the millis() function in Arduino, coupled with a tilt sensor, enhances the efficiency and versatility of the digital timer. The shift from delay() to millis() mitigates the blocking issues associated with the former, making the timer more responsive. The inclusion of a tilt sensor introduces a hands-free aspect to the timer, catering to diverse user scenarios. This study showcases the significance of adopting non-blocking mechanisms in Arduino programming for improved functionality and responsiveness in time-sensitive applications. The digital timer with millis() function and tilt sensor integration presents a

robust solution suitable for a range of projects where accurate timing and user-friendly interaction are paramount.

Reference(s):

1. https://www.arduino.cc/.
2. ATMega328 manual
3. https://www.avrfreaks.net/forum/tut-c-newbies-guide-avr-timers

http://maxembedded.com/2011/06/avr-timers-timer0/